UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 2.7 – Mapa *thread-safe*

1. Crie uma classe de aplicação com uma estrutura de dados (coleção) para simular um recurso compartilhado.

Classe MapaConcorrente.java

```java
public class MapaConcorrente {

	/* ConcurrentHashMap:
	 * sincroniza porções do mapa que estão tendo acesso concorrente.
	 * um pouco mais lenta, porém é thread-safe.
	 *
	 * */

	static Map<Integer, String> mapa = new ConcurrentHashMap<>();

	public static void main(String[] args) throws InterruptedException {


	}

}
```

2. Crie uma classe para implementar uma thread que acessa este recurso.

Classe ThreadMapaConcorrente.java

```java
public class ThreadMapaConcorrente implements Runnable{

	@Override
	public void run() {

		Integer id = (int) Thread.currentThread().getId();
		String nome = Thread.currentThread().getName();
		MapaConcorrente.mapa.put(id, nome);
		System.out.println(nome + " inseriu no mapa!");

	}

}
```

3. Na aplicação:

Classe MapaConcorrente.java

```java
public class MapaConcorrente {

	/* ConcurrentHashMap:
	 * sincroniza porções do mapa que estão tendo acesso concorrente.
	 * um pouco mais lenta, porém é thread-safe.
	 *
	 * */

	static Map<Integer, String> mapa = new ConcurrentHashMap<>();

	public static void main(String[] args) throws InterruptedException {

		ThreadMapaConcorrente tmc = new ThreadMapaConcorrente();

		Thread t0 = new Thread(tmc);
		Thread t1 = new Thread(tmc);
		Thread t2 = new Thread(tmc);

		t0.start();
		t1.start();
		t2.start();

		// aguarda as threads executarem sem esperarem entre si
		Thread.sleep(300);

		System.out.println(mapa);

	}

}
```